
SWHV CCN4 WCS and HPC Validation Note

SWHV-ROB-TN-001-CCN4 v3.1

SWHV Team



SWHV-ROB-TN-001-CCN4 - Version 3.1 - 2026-09-03

Contents

1	Summary	2
2	Validation approach	4
2.1	JHV behavior modeled by the Python validator	4
2.2	Python/Astropy validation scope	6
2.3	Running the validation tools	7
2.4	Direct Java/Python metadata comparison	9
2.5	GLSL/Electron validation setup	11
2.6	Failure checks and acceptance limits	12
2.7	Astropy-based validation modes	13
2.8	Internal JHV comparison modes	16
3	Results	18
3.1	FITS projections	18
3.2	Astropy validation of JHV HPC rendering	20
3.3	Formal-TAN versus JHV HPC	21
3.4	Simple-TAN versus JHV HPC	22
3.5	CAR surface-map validation	23
3.6	CEA surface-map validation	24
3.7	Dense surface-map render compare	25
3.8	GLSL/Electron validation results	25
3.8.1	Rendered synthetic-source results	27
4	Appendix A: Theoretical Orthographic vs HPC discrepancy	30
5	Appendix B: Idealized TAN small-angle discrepancy relative to HPC	33

List of Figures

4.1	Orthographic vs HPC discrepancy across the projected solar disk at 1 AU	31
4.2	Maximum Orthographic vs HPC discrepancy versus observer distance	32
5.1	Maximum idealized TAN small-angle discrepancy relative to HPC versus observer distance	34

id: 38c4c982cb602a473a529d1691e4eb543c71d0dd

1 Summary

This note describes the zenithal WCS and `HPC` work in JHV and the validation against Astropy, Java/Python metadata checks, and direct GLSL checks. It also includes validation cases for surface maps described by FITS WCS axes `CRLN-CAR` / `CRLT-CAR` and `CRLN-CEA` / `CRLT-CEA`. The main focus remains the zenithal projections and their use in `HPC`. For the general FITS/WCS header conventions used throughout, the primary reference is Greisen and Calabretta, “Representations of world coordinates in FITS” (A&A 395, 1061-1075, 2002).

JHV now has a new `TAN` path, `ARC`, `AZP`, six-term `ZPN`, and support for `CRLN-CAR` / `CRLT-CAR` and `CRLN-CEA` / `CRLT-CEA` surface maps, using the FITS celestial-coordinate projection formalism of Calabretta & Greisen (A&A 395, 1077-1122, 2002). Hereafter, `formal-TAN` denotes this FITS-compliant `TAN` formulation in JHV.

The JHV `TAN` implementation of previous versions is hereafter denoted as `simple-TAN`. It treats the orthographic surface point’s planar (x, y) coordinates as a small-angle approximation to helioprojective coordinates. It then feeds those coordinates directly into the linear image transform, instead of first converting the 3D point to helioprojective angles and then applying the `formal-TAN` world-to-plane projection.

An `HPC` projection mode was also added to JHV and is validated here. Its solar-coordinate interpretation follows Thompson, “Coordinate systems for solar image data” (A&A 449, 791-803, 2006). Because `HPC` is fundamentally tied to a specific observer viewpoint, it is not a natural fit for the multi-viewpoint aspect of JHV.

The `CAR` and `CEA` cases in this note are surface-map cases: they describe solar longitude/latitude on the sphere rather than observer-centered image geometry. They are a natural fit for `Latitudinal` and for wrapping the visible sphere in `Orthographic`, but not for `HPC`, `RadialWarp`, or `RectWarp`.

JHV accepts two CEA second-axis conventions. When `CUNIT2` is present, the linear transform is angular and follows FITS WCS/wcslib. Historical normalized maps that omit `CUNIT2` are kept compatible by interpreting their second plane coordinate directly as `sin(latitude) / PV2_1`. This missing-unit convention is a documented JHV compatibility assumption: FITS WCS and Astropy otherwise default a missing celestial unit to degrees.

The supported solar WCS subset assumes the canonical native poles present in the validation data. Observer-image `HPLN/HPLT` projections use `LONPOLE=180°`, whereas `CRLN/CRLT` surface maps use `LONPOLE=0°` and `LATPOLE=90°`. JHV does not implement arbitrary FITS celestial/native pole rotations, so a noncanonical surface `CRVAL2` cannot be treated independently of those pole parameters.

Heliospheric imager datasets often use `AZP` or `ZPN` projections. For these datasets, these tests support the correctness of the `HPC` WCS and sampling path. That conclusion does not extend automatically to

Orthographic mode, where wide-angle heliospheric images may still not display satisfactorily because their integration into the JHV 3D viewing model remains a separate problem.

The validation uses three checks. First, a Python model of the JHV WCS, reprojection, and sampling path is compared against Astropy wherever the result is fully defined by WCS. Second, the WCS metadata derived by the JHV Java code is compared with the same Python metadata model on the FITS test files. Third, the JHV GLES3 GLSL shader sources are run through an Electron/WebGL2 runner on Metal/ANGLE and SwiftShader. The FITS examples include COR2, CCOR2, HI1, HI2, AIA, EU1, and WISPR data.

Main conclusions:

- **formal-TAN**, **ARC**, **AZP**, and six-term **ZPN** are validated against Astropy, including the tested inverse mappings.
- the HI test files cover non-slanted **AZP** ($\gamma = 0$). The generated production-fragment cases also test slanted **AZP** with $\gamma = 12^\circ$.
- JHV **HPC** rendering is validated against Astropy for the WCS and sampling path covered by this note.
- the **CAR** and explicitly unit-labeled **CEA** cases are validated directly against Astropy for the source WCS interpretation and sampling path. The normalized missing-unit CEA case is first converted to its equivalent angular CEA header for the Astropy comparison.
- the Electron/WebGL2 checks run the JHV GLSL coordinate path directly. The recorded Metal/ANGLE coordinate results below reach about 0.002 source-image pixels, but do not establish a maximum for the expanded suite.
- **formal-TAN** in **Orthographic** mode and **HPC** are not the same display geometry, even with the same observer viewpoint and $\text{dragRotation} = 0$. The measured discrepancy is consistent with the geometric difference derived in Appendix A.
- on the sample TAN files, **simple-TAN** remains very close to the **HPC** display geometry. It is kept in JHV **Orthographic** mode for data designated as **TAN** in the metadata.

These checks validate the specified WCS and rendering paths, not the design of JHV's viewing modes, playback policy, or overlays. In particular, the position panel at the bottom of the JHV window reports coordinates derived from the mouse pointer and display geometry, not coordinates read back from the active image WCS. In **HPC**, **Latitudinal**, **RadialWarp**, and **RectWarp**, they follow the matching JHV display projection. In **Orthographic**, they come from the inferred 3D scene point and do not, in general, reflect the image WCS.

2 Validation approach

For the Astropy-based validation modes in this note, the Python validator provides a Python/CPU model of the relevant JHV WCS, reprojection, and source-sampling logic. Beyond derived WCS quantities, it mirrors key Java/GLSL renderer conventions used in the tested paths, including the split between zenithal source-image and surface-map sampling, direct world-to-lon/lat projection for `CAR` and `CEA`, wrapped longitude sampling for full-width surface maps, the FITS-to-texture row flip, bottom-origin screen coordinates, texcoord/discard rules, and GLSL-style linear texture sampling. These comparisons check the Python model against Astropy, not the running JHV renderer. Separate tests compare Java metadata derivation with that model and execute the actual GLSL sources through Electron/WebGL2.

Astropy does not implement JHV's `Orthographic` display mode. A display pixel first selects a point on the solar sphere or the off-limb plane, expressed in the image observer's frame. The validator constructs that geometry and uses Astropy only to convert the resulting viewing angles through the input FITS WCS. The `simple-TAN` shortcut is checked separately as an affine mapping of the point's x/y coordinates. Agreement with Astropy therefore validates the downstream WCS operation, not JHV's choice of sphere or plane.

The rest of this section follows the same split:

- Python/Astropy validation of the WCS and sampling model
- Java/Python comparison of the derived metadata used by JHV
- Electron/WebGL2 execution of the JHV GLES3 GLSL shader sources

2.1 JHV behavior modeled by the Python validator

For each screen pixel, JHV first determines the matching point on the display geometry. In `HPC` mode, that geometry is the `HPC` display plane. In `Orthographic` mode, it is the 3D solar scene used by the ortho renderer. From that display point, JHV derives the helioprojective coordinates needed to evaluate the image WCS. For zenithal FITS data in the formal WCS paths, this means applying the relevant world-to-plane projection, such as `TAN`, `ARC`, `AZP`, or `ZPN`, using the reference direction given by `CRVAL`. The linear transform (`CD` or `CDELTA` with `PC/CROTA`) and `CRPIX` then locate the point in the source image. The screen pixel is produced by sampling the source image at those coordinates with interpolation. In this sense, the WCS projection determines how image coordinates relate to observer geometry, while the JHV rendering mode determines which display geometry is sampled before that WCS mapping is evaluated.

The sampling pipeline described above, and the point at which `simple-TAN` departs from the formal WCS path, can be summarized as follows:

```
formal WCS:
screen pixel
-> display geometry point
-> helioprojective coordinates
-> WCS world-to-plane step
-> linear WCS terms
-> source-image coordinates
-> interpolated image sample

simple-TAN:
screen pixel
-> display geometry point
-> (x,y) coordinates in the image-view plane
-> linear WCS terms
-> source-image coordinates
-> interpolated image sample
```

In `simple-TAN`, the `(x,y)` pair refers to the display point's coordinates in the image-view plane, i.e., the plane perpendicular to the viewing direction in the image observer frame. Those coordinates are used directly as a small-angle approximation to the helioprojective viewing angles, instead of first converting the 3D point to helioprojective coordinates and then applying the `formal-TAN` projection step.

Included in the Python validator:

- FITS metadata parsing into the same effective quantities JHV uses:
 - `CRPIX`
 - `CRVAL`
 - `CD`, or `CDEL` with `PC/CROTA`
 - `DSUN_OBS`
 - `PV2_0..PV2_5`
- the shared WCS projection math for:
 - `TAN`
 - `ARC`
 - `AZP`
 - six-term `ZPN` (primary branch only)
 - `CAR`
 - `CEA`
- the `HPC` image sampling path:
 - screen `HPC` coordinate -> helioprojective -> WCS plane -> source pixel
- the raw image-footprint `HPC` bounds and the centered `HPC` display bounds used in these validation runs
- the `ARC` source-image WCS interpretation used for the tested `HPLN-ARC` / `HPLT-ARC` case
- the `CAR` and `CEA` source WCS interpretation used for `CRLN-CAR` / `CRLT-CAR` and `CRLN-CEA` / `CRLT-CEA` surface maps
- direct comparison between the `formal-TAN` path in `Orthographic` mode and `HPC` for the specific no-rotation comparison mode.

The geometry model accepts camera and source-view rotations, and the production-fragment tests exercise fixed rotations, panning, and zooming. They check the shader's response to those inputs, not the interactive Java code that derives them from mouse actions or changing layers.

Outside this validation scope:

- the live camera and layer lifecycle that produces `dragRotation`, `cameraDiff`, and other shader inputs
- differential diff-image alignment between two arbitrary live layers beyond the specific per-image `deltaT` reprojection modeled in the shader path
- `deltaCROTA`, `deltaCRVAL1`, `deltaCRVAL2`
- mouse-position reporting or synthetic overdrawing such as annotations, grid overlays, and similar on-screen constructs
- playback/view policy effects such as dynamic refitting of the visible `HPC` box beyond the centered bounds logic explicitly reproduced by the Python validator, or any other transient user-driven viewing state
- `CAR/CEA` display-policy choices beyond the validated source-WCS interpretation and sampling path
- Java overlay-only behavior such as:
 - viewpoint-space projection of external overlay points in `HPC`
 - visible-hemisphere clipping of `HPC` overlay emission
 - transformed annotation drawing/picking behavior
- full OpenGL rasterization state, blending, and UI behavior

The Python validator checks:

- whether the reprojection and sampling math matches Astropy
- whether the `HPC` sampling map matches Astropy
- whether the `CAR/CEA` source WCS interpretation and sampling path match Astropy
- whether the `formal-TAN` path in `Orthographic` mode and `HPC` sample the same source pixels over the same rendered comparison frame.

2.2 Python/Astropy validation scope

Astropy can validate the parts of the mapping that are fully defined by WCS.

Here, Astropy is the external reference for:

- forward samples
 - JHV-style world/helioprojective -> WCS plane -> pixel
 - checked against `astropy.wcs.WCS.wcs_world2pix(...)`
- full pixel-center validation
 - every source-image pixel center is mapped through the JHV path and checked against Astropy
- inverse `TAN`, `ARC`, `AZP`, and `ZPN`
 - plane -> helioprojective, compared directly with Astropy's inverse WCS
 - `ZPN` is checked only on the supported primary monotonic branch
- bounded `HPC` render comparison

- the same [HPC](#) screen grid is run through the JHV sampling path and through Astropy [world](#) -> [pixel](#)
 - the script compares the resulting source-pixel centers and also writes diff images
- centered [HPC](#) bounds reporting
 - the script reports both the raw image-footprint bounds and the centered display bounds used by the JHV [HPC](#) screen mapping
- full pixel-center validation for [CAR](#)
 - using the effective linear transform ([CD](#), or [diag\(CDELT1, CDELT2\) * PC](#)) together with the native [CRLN-CAR](#) / [CRLT-CAR](#) axis semantics
- inverse [CAR](#)
 - plane -> world lon/lat, compared directly with Astropy's inverse WCS
- full pixel-center validation for [CEA](#)
 - using the effective linear transform ([CD](#), or [diag\(CDELT1, CDELT2\) * PC](#)) together with the native [CRLN-CEA](#) / [CRLT-CEA](#) axis semantics
 - headers with an explicit [CUNIT2](#) are passed to Astropy unchanged
 - the supported normalized missing-unit convention is canonicalized to an equivalent degree-valued second row before constructing the Astropy WCS
- inverse [CEA](#)
 - plane -> world lon/lat, compared directly with Astropy's inverse WCS

For each inverse test, Astropy's forward WCS first establishes valid plane inputs. The Python implementation's inverse result is compared with `astropy.wcs.WCS.wcs_pix2world(...)`. A separate round-trip check applies the implementation's forward projection to its inverse result and compares it with the original plane input.

Not compared against Astropy:

- direct comparison between the [formal-TAN](#) path in [Orthographic](#) mode and [HPC](#)
 - this compares the two display mappings within the Python model

2.3 Running the validation tools

Validation follows the rendering paths that matter in JHV:

1. [HPC](#) rendering is the primary WCS test. Astropy validates the FITS WCS portion, while JHV's bounds, pixel centers, viewport mapping, and source-image discard rules define the rendered population.
2. Latitudinal [CAR/CEA](#) rendering is the primary cylindrical-WCS test.
3. Orthographic geometry is checked with an independent ray/sphere/plane construction, including rotated views and off-limb points. Warp geometry is compared with the corresponding CPU model. Astropy checks the downstream FITS WCS operation in each case, not the display geometry itself.

4. Metal/ANGLE and SwiftShader are each compared with the appropriate reference. A direct back-end comparison supplements those checks, but cannot replace comparison with an independent reference.

Raw coordinate differences outside JHV's rendered source-image bounds are not errors. Conversely, sparse coordinate/discard differences at an image edge are reported separately. The sampled-texture checks determine their actual output effect by comparing the rendered value, including transparent-black discarded pixels, with an independently mapped reference value.

The main Python validator is:

- `extra/test/validate_jhv_wcs_against_astropy.py`

The documented test set can also be run as a suite with:

- `extra/test/run_jhv_wcs_hpc_validation_suite.py`

The suite includes the Java/Python metadata-comparison check:

- `extra/test/compare_java_metadata_to_validator.py`

Run the prioritized CPU, GLSL-syntax, and Java-metadata suite with:

```
python3 extra/test/run_jhv_wcs_hpc_validation_suite.py
```

Add the focused Metal/ANGLE and SwiftShader shader checks with:

```
python3 extra/test/run_jhv_wcs_hpc_validation_suite.py --include-electron
```

The helper checks execute the production coordinate and WCS sampling helpers over JHV's viewport bounds and render a synthetic R16F texture rather than stopping at raw coordinates. Separate focused checks exercise differential rotation, independently populated primary and secondary WCS uniform-block slots, and the user-sector, metadata-sector, radial, and cutoff display masks. These helper checks do not execute complete production fragments. The suite also runs the color, coordinate, and depth tests in `test_wcs_rendering.py`, described below, using the production fragment paths. Neither group reproduces the application's entire frame-compositing sequence. In the helper checks, `frame_sample_*` measures the complete viewport, including visibility differences. `joint_sample_*` measures rendered values only where both implementations consider the source visible, and is the sample-accuracy gate. `texture_interpolation_*` separately compares GPU and CPU filtering at the GPU coordinate without checking coordinate correctness. The CPU sampler uses the same half-float source values and clamp-to-edge filtering as the WebGL/JHV texture.

Both backends run the production-fragment tests across all six supported FITS projections in Orthographic mode, along with the applicable HPC, latitudinal, and warp cases. A combined HPC projection check also compares each backend with Astropy and then directly with the other backend. The older `electron_swiftshader_all_modes` helper diagnostic remains available through `--only` for investigating larger SwiftShader warp errors on the FITS test files. It is separate from the production-fragment tests and is not included among the pass/fail checks in `--extended`.

Run only those focused shader checks with:

```
python3 extra/test/run_jhv_wcs_hpc_validation_suite.py --electron-only
```

Successful suite runs write generated PNGs and Electron float buffers to temporary storage and delete them after evaluation. Failure artifacts are retained and their temporary path is printed. Expensive CPU/Astropy rendering diagnostics are reduced to 512 by 512 in the routine core. Electron checks keep the resolution declared by each case, normally 128 by 128 for sampled-texture comparisons. Use `--extended` for the complete historical catalog at its documented larger sizes, or `--only <name> [...]` for selected diagnostics.

Run the Python validator with:

```
python3 extra/test/validate_jhv_wcs_against_astropy.py <fits-file> [mode]
```

The Python validator uses four comparison regions:

- full image frame:
 - the actual FITS image pixel grid
- full rendered comparison frame:
 - the full square comparison grid used by the internal `Orthographic/HPC` tests
- bounded `HPC` screen domain:
 - a finite rendered `HPC` box chosen for the validation run
- dense surface-map render grid:
 - a rendered longitude/latitude grid used for the additional `CAR/CEA` surface-map sampling checks

2.4 Direct Java/Python metadata comparison

The Python validator described above is intentionally independent of the Java metadata code. Astropy agreement alone does not prove that Java interprets the FITS headers in the same way as the Python model.

To check that point, this note also uses a small Java-side metadata dumper:

- `extra/test/JHVMetadataDump.java`

and a Python driver that compiles and runs that helper:

- `extra/test/compare_java_metadata_to_validator.py`

The comparison driver:

- compiles the JHV Java code and the `JHVMetadataDump.java` helper
- opens FITS files from the documented validation suite
- instantiates `FitsMetaData` (the JHV class that interprets the FITS metadata into the derived image/WCS quantities used by the program) on the Java side
- extracts the derived WCS metadata quantities produced by JHV Java code
- compares them field-by-field against the Python validator's `build_jhv_meta(...)`

The comparison covers the FITS/HDU cases from the documented validation suite, including:

- [TAN](#)
- [ARC](#)
- [AZP](#)
- [ZPN](#)
- [CAR](#)
- [CEA](#)

The compared derived quantities include:

- pixel size:
 - [pixel_width](#)
 - [pixel_height](#)
- FITS reference-pixel conversion to the JHV/OpenGL convention:
 - [crpix1_gl](#)
 - [crpix2_gl](#)
- effective per-axis source scale:
 - [arcsec_per_pixel_x](#)
 - [arcsec_per_pixel_y](#)
- normalized internal scaling:
 - [unit_per_arcsec](#)
 - [unit_per_pixel_x](#)
 - [unit_per_pixel_y](#)
 - [plane_units_per_rad](#)
- normalized WCS reference values:
 - [crval_internal_x](#)
 - [crval_internal_y](#)
 - [image_to_plane](#)
- projection family and projection parameters:
 - [projection](#)
 - [pv2](#)
- image bounds derived from that metadata:
 - the [HPC](#) footprint bounds for observer-centered projections
 - the radial bound used by [RadialWarp](#) and [RectWarp](#)
 - the Sun shift used by radius-aware image filters

This comparison catches Java-side metadata interpretation drift that may not be visible from the Astropy agreement alone.

In the code state documented here, this Java/Python metadata comparison agrees on all cases used by the comparison script.

The comparison script also generates small temporary FITS cases for unequal and negative axis scales with a non-orthogonal `PC`, partial `PC` defaults, full and partial `CD` matrices, omitted `CDELTA`, `PC/CD/CROTA` precedence, mixed angular units, unit-aware surface-map reference longitude, and `PC/CD` matrices on `CAR` and both explicitly unit-labeled and normalized missing-unit `CEA` surface maps. All generated cases compare Java and Python metadata. All except two also compare against Astropy: the missing-observer-unit case checks JHV's compatibility default rather than Astropy's unit interpretation, and the LASCO case checks JHV's policy of suppressing the image WCS transform.

One caveat remains for the `observer_distance` field when `DSUN_OBS` is absent:

- on the JHV side, the fallback observer distance comes from the runtime SPICE ephemeris path
- on the Python side, the fallback uses Astropy solar ephemerides

Those two fallback ephemerides do not yield exactly the same observer distance. On the tested missing-`DSUN_OBS` cases, the difference is tiny, so the direct comparison accepts a small tolerance for `observer_distance` in that case. This does not affect the conclusion about WCS metadata interpretation.

2.5 GLSL/Electron validation setup

The Python model evaluates the formulas in double precision, using projection coefficients rounded to float32 to match JHV's shader inputs. Astropy uses the original FITS coefficients, so comparisons also include the expected effect of that rounding. The production renderer evaluates the formulas in GLES3 GLSL with float32 arithmetic, texture sampling, and backend-specific shader compilation.

To test the shader path outside the application, the suite also includes an Electron/WebGL2 runner:

- `extra/test/validate_jhv_wcs_with_electron.py`
- `extra/test/electron_webgl_runner/runner.html`

The Electron runner:

- starts the configured Electron executable (`--electron` or `JHV_ELECTRON`)
- creates a WebGL2 context
- compiles the JHV shader sources from `resources/glsl`
- verifies the reflected sizes and active-member offsets of the JHV `ImageBlock`, `ScreenBlock`, and `DisplayBlock`, then binds matching UBO data
- renders diagnostic or production shader variants into an `RGBA32F` framebuffer
- reads the float framebuffer back to Python
- compares shader-generated texture coordinates and samples against the Python/Astropy reference path

The helper diagnostics expose texture coordinates and sampled values for numerical comparison. The older “color smoke” checks establish only that the shader produces finite output in range. An opaque black image can pass such a check, so these runs are not evidence of correct colors or subtraction.

`extra/test/test_wcs_rendering.py` adds quantitative production-fragment tests:

- all six supported FITS projections in JHV’s orthographic mode, with sphere and off-limb samples, camera rotation, panning, and zooming;
- observer-image projections in HPC, latitudinal, and both warp modes, plus CAR/CEA surface maps in latitudinal mode;
- known R8 and R16F textures, a discontinuous colored LUT, brightness, and a premultiplied layer tint;
- difference rendering with different source images and different reference pixels, so reading the primary texture or WCS twice cannot pass;
- depth probes with and without the production solar sphere, checking both on-disk depth and the off-limb background depth.

Coordinate checks retain the production fragment `main()` and replace only its final color calculation. Color and depth checks execute the unchanged production shader. Color comparisons allow the shader’s bounded dithering and half-precision filtering error, including its amplification by subtraction. These tests run on both Metal and SwiftShader. They exercise the GLSL sources and their uniform layouts, but do not replace tests of Java image decoding, cache ownership, or the application’s complete layer-compositing sequence.

The CCOR2 fixture `CCOR2_1A_20260902T211514_V00_NC.fits` preserves the original compressed FITS file, with its `ZPN` image in HDU 1. The core suite checks its Java/Python metadata, inverse WCS, and HPC and orthographic sampling on both backends. Its orthographic case guards against applying the simple-TAN reference shortcut to a non-TAN image. The inverse check uses an explicit $1e-6$ -degree tolerance to allow for JHV’s float32 polynomial coefficients, which introduce a $2.36e-7$ -degree difference from the original FITS precision.

Two Electron backends were used for the results below:

- **default:** Electron/ANGLE on macOS Metal, reported by WebGL as `ANGLE Metal Renderer: Apple M3 Pro`
- **swiftshader:** Electron/ANGLE on SwiftShader, a Vulkan software rendering implementation

The Metal backend corresponds to JHV’s macOS rendering path. SwiftShader gives a second backend run, with larger coordinate differences in some projection paths because its transcendental/math behavior differs.

2.6 Failure checks and acceptance limits

`extra/test/test_wcs_validation.py` deliberately supplies incorrect results to the validators. It checks rejection of wrong inverse coordinates, nonfinite values, missing pixels, missing Astropy comparisons, GL errors, incorrect production colors, and out-of-order failures in the parallel suite runner. Its geometry checks also distinguish the sphere, plane, and back-side cases.

Inverse comparisons require valid reference samples and enforce an angular and projection-plane round-trip limit of $1e-7$ degrees, configurable with `--max-inverse-error-deg`. Pixel-error limits are no longer silently increased inside GPU evaluators. The suite explicitly passes `--max-error-px 1.0` for the existing SwiftShader differential-rotation case, whose latitudinal error is about 0.81 source pixel. Direct invocations respect a stricter requested limit.

HPC GPU and difference self-checks permit crop-boundary disagreements only within the requested coordinate tolerance of an image edge, capped at 0.1% of the viewport with a minimum budget of one pixel. The new production cases use a tighter 0.05-pixel edge band and the same population cap. Interior omissions always fail. The backend comparison additionally limits mean full-frame intensity error to the sample tolerance plus 0.001 for that bounded edge population. Maximum edge error remains reported, because a single missing bright pixel can have a large maximum even when the coordinate discrepancy is subpixel.

Runs that generate previews, compare deliberately different display mappings, or explicitly request `--report-validity-mismatches` are labeled diagnostics and counted separately from correctness assertions. A diagnostic still fails on an execution error, but completing it is not proof of rendering accuracy.

2.7 Astropy-based validation modes

1. Forward WCS sampled validation

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \
  extra/test/data/20241224_194245_d4c2A.fts
```

This mode reports forward WCS errors on sampled points. For `TAN`, `ARC`, and `AZP`, the samples are generic world/helioprojective points. For `ZPN`, `CAR`, and `CEA`, the Python validator uses pixel-derived world samples so that edge, pole, and seam cases are covered more systematically. It is not tied to the displayed solar disk or to the full image frame.

It reports:

- `projection_max_error_internal`
- `pixel_center_max_error_px`

2. Full pixel-center validation

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \
  extra/test/data/20250622_000831_s4h1A.fts \
  --all-pixels
```

This mode checks the full FITS image grid against Astropy and reports the worst pixel-center error.

For the `CAR` and `CEA` cases, the same mode also validates surface maps against Astropy over the full source image grid. In those cases, the Python validator uses the effective linear transform `CD`, or `diag(CDELT1, CDELT2) * PC`, not bare `CDELT`, and keeps the original axis types (`CRLN-CAR` / `CRLT-CAR` or `CRLN-CEA` / `CRLT-CEA`) when constructing the Astropy comparison WCS. Explicitly unit-labeled CEA headers are passed through unchanged. For normalized CEA headers with no `CUNIT2`, it converts only the second linear-transform row to the equivalent angular values and sets `CUNIT2=deg`, because Astropy otherwise correctly applies the FITS default of degrees to the original missing unit.

3. Inverse TAN

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \
  extra/test/data/sample.171.fits \
  --hdu 1 \
```



```
--inverse-tan
```

This mode validates inverse TAN on sampled projection-plane points. It is not an on-disk or full-image-grid test.

It validates:

- TAN plane -> helioprojective
- round-trip error

4. Inverse ARC

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/PUNCH_L3_CAM_20260425001600_v0k.fits \  
--hdu 1 \  
--inverse-arc
```

This mode validates inverse ARC on sampled projection-plane points. It is not an on-disk or full-image-grid test.

It validates:

- ARC plane -> helioprojective
- round-trip error

5. Inverse AZP

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/20250622_000831_s4h1A.fts \  
--inverse-azp
```

This mode validates inverse AZP on sampled projection-plane points. It is not an on-disk or full-image-grid test.

It validates:

- AZP plane -> helioprojective
- round-trip error

6. Inverse primary-branch ZPN

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/psp_L3_wispr_20231227T150704_V1_2222.fits \  
--inverse-zpn
```

This mode validates inverse ZPN on sampled projection-plane points. It is not an on-disk or full-image-grid test.

It validates:

- ZPN plane -> helioprojective
- round-trip error on the primary monotonic branch

7. HPC render comparison

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \
  extra/test/data/20241224_194245_d4c2A.fits \
  --hpc-render-compare \
  --render-size 2048
```

This mode runs over a bounded HPC screen domain and sends the same rendered screen grid through:

- the JHV HPC sampling path
- Astropy world -> pixel

and writes:

- *_hpc_jhv.png
- *_hpc_astropy.png
- *_hpc_diff.png

under:

- extra/test/out

It also reports:

- raw_bounds_deg
- bounds_deg
- pixel_center_max_error_px
- pixel_center_rms_error_px

Interpretation:

- the script reports absolute pixel errors directly
- HI2 can show a slightly larger absolute px error because AZP magnification near the finite valid edge becomes extremely large

8. Inverse CAR

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \
  extra/test/data/sunerf_map.fits \
  --inverse-car
```

This mode validates inverse CAR on sampled projection-plane points. It is not an on-disk or full-image-grid test.

It validates:

- CAR plane -> world lon/lat
- round-trip error

9. Inverse CEA

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \
  extra/test/data/mrzqs260301t2314c2308_169.fits \
  --inverse-cea
```

This mode validates inverse CEA on sampled projection-plane points. It is not an on-disk or full-image-grid test.

It validates:

- CEA plane -> world lon/lat
- round-trip error

2.8 Internal JHV comparison modes

1. formal-TAN vs JHV HPC

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/sample.171.fits \  
--hdu 1 \  
--ortho-vs-hpc-screen-compare \  
--render-size 4096
```

This mode compares, for TAN data:

- the `formal-TAN` path in `Orthographic` mode
- the JHV HPC display sampling path

over the full rendered comparison frame, using a comparison grid at the native image resolution of the tested file.

It writes:

- `*_ortho_screen.png`
- `*_hpc_screen.png`
- `*_ortho_vs_hpc_diff.png`

Interpretation:

- this is not an Astropy comparison
- it measures whether the `formal-TAN` path in `Orthographic` mode and `HPC` define the same on-screen geometry over that full rendered frame

2. simple-TAN vs JHV HPC

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/sample.171.fits \  
--hdu 1 \  
--compare-initial-tan-vs-hpc
```

This mode compares, for TAN data:

- `simple-TAN`
- the `HPC` display sampling path

over the full rendered comparison frame, using a comparison grid at the native image resolution of the tested file.

It writes:

- `*_initial_tan_screen.png`
- `*_hpc_screen_from_initial_tan_compare.png`
- `*_initial_tan_vs_hpc_diff.png`

It also reports:

- `initial_tan_vs_hpc_max_px`
- `initial_tan_vs_hpc_rms_px`

Interpretation:

- this is an internal JHV comparison for `TAN`
- it measures how closely `simple-TAN` in `Orthographic` mode matches the `HPC` display geometry over that full rendered frame

3. `simple-TAN` vs `formal-TAN` over the full image frame

```
python3 extra/test/validate_jhv_wcs_against_astropy.py \  
extra/test/data/sample.171.fits \  
--hdu 1 \  
--compare-initial-tan-image-frame
```

This mode compares:

- `simple-TAN`
- `formal-TAN`
- Astropy as the WCS reference

over the full FITS image frame at native image resolution.

It writes:

- `*_initial_tan_image_frame.png`
- `*_formal_tan_image_frame.png`
- `*_initial_vs_formal_tan_image_frame_diff.png`

Interpretation:

- this is an implementation-comparison mode for `TAN` only
- it compares `simple-TAN` and `formal-TAN` against Astropy over the actual FITS image frame and shows where the two JHV paths diverge

3 Results

The measurements below record earlier runs on the named FITS files and configurations. They illustrate the comparisons, but are not a consolidated report of the expanded suite described above. In particular, their maxima are not suite-wide accuracy limits.

Forward and rendering comparisons report source-image pixel errors, the quantity measured directly by the validators. The accompanying mas or arcsec values help put their magnitude in familiar units. Each *nominal-scale equivalent* is the pixel error multiplied by the image's nominal pixel scale, not an angular separation measured on the sky, because this conversion does not account for the nonlinear WCS projection. Inverse results instead report the larger absolute longitude or latitude residual, with longitude wrapping accounted for. Those angular-coordinate residuals and the theoretical angles in the appendices are not pixel-scale conversions.

3.1 FITS projections

These tests compare the Python model of JHV's projection formulas against Astropy as the external WCS reference.

1. `formal-TAN` agrees with Astropy on the tested samples.
 - `JHV world -> helioprojective -> TAN plane -> pixel`
 - `TAN plane -> helioprojective angles`
 - matches Astropy to numerical precision
 - measured results:
 - forward sampled check on `20241224_194245_d4c2A.fits`:
 - * `6.821210e-12 px max`
 - * nominal-scale equivalent: `1.00e-7 mas max`
 - * `projection_max_error_internal=1.012523e-13`
 - inverse TAN on `sample.171.fits`:
 - * `2.05e-7 mas max`
 - * `roundtrip_plane_max_error_internal=8.881784e-16`
2. `formal-TAN` is substantially more accurate than `simple-TAN`.
 - native-resolution full-image-frame comparison against Astropy on the same TAN samples gives:
 - `sample.171.fits (1.009 AU)`:
 - * `simple-TAN: 3.674571 px max, 1.864828 px RMS (nominal-scale equivalents: 2.20 arcsec max, 1.12 arcsec RMS)`

- * `formal-TAN`: 4.357389e-09 `px` max, 1.084205e-10 `px` RMS (nominal-scale equivalents: 2.61e-6 `mas` max, 6.49e-8 `mas` RMS)
- `solo_L2_eui-fsi174-image_20251002T150055171_V00.fits` (0.448 AU):
 - * `simple-TAN`: recorded nominal-scale equivalents of 11.42 `arcsec` max and 2.46 `arcsec` RMS
 - * `formal-TAN`: 3.109335e-10 `px` max, 1.292454e-11 `px` RMS (nominal-scale equivalents: 1.38e-6 `mas` max, 5.74e-8 `mas` RMS)
- `20241224_194245_d4c2A.fts` (0.967 AU):
 - * `simple-TAN`: recorded nominal-scale equivalents of 2.40 `arcsec` max and 0.252 `arcsec` RMS
 - * `formal-TAN`: 7.116796e-11 `px` max, 4.363033e-12 `px` RMS (nominal-scale equivalents: 1.05e-6 `mas` max, 6.42e-8 `mas` RMS)
- for `simple-TAN`, the maximum is set by the on-disk region in all three cases. The RMS values are calculated over the full image frame, which includes both the on-disk sphere region and the off-limb flat-plane region used by the `Orthographic` renderer outside the solar disk. `formal-TAN` remains at numerical precision across the full frame.
- this does not contradict Appendix B. The `simple-TAN` small-angle-approximation error $\tan(a) - a$ grows with angular distance from the boresight, but this comparison does not measure that error directly everywhere. On the solar disk, `Orthographic` samples from the solar sphere. Outside the solar disk, `Orthographic` uses a flat image-view plane to sample the WCS projection instead. The off-limb part of the comparison is not a direct measurement of the `simple-TAN` small-angle-approximation error alone.

3. ARC agrees with Astropy for the PUNCH test file.

- JHV `world` -> `helioprojective` -> ARC plane -> `pixel`
- ARC plane -> `helioprojective` angles
- matches Astropy to numerical precision on:
 - `extra/test/data/PUNCH_L3_CAM_20260425001600_v0k.fits`
- measured results:
 - forward sampled check on the same file:
 - * 3.278956e-09 `px` max
 - * nominal-scale equivalent: 2.66e-4 `mas` max
 - * `projection_max_error_internal`=2.785048e-10
 - full pixel-center check on the same file:
 - * 2.751221e-10 `px` max
 - * nominal-scale equivalent: 2.23e-5 `mas` max
 - inverse ARC on the same file:
 - * 2.05e-7 `mas` max
 - * `roundtrip_plane_max_error_internal`=2.421718e-11

4. AZP agrees with Astropy for the non-slanted HI test files.

- JHV `world` -> `helioprojective` -> AZP plane -> `pixel`

- `AZP plane` -> `helioprojective angles`
- matches Astropy to numerical precision
- the tested HI files are non-slanted: `PV2_2` is absent, so `gamma = 0`
- slanted `AZP` is covered separately by the generated production-fragment cases with `gamma = 12`°, not by these HI test files
- measured results:
 - full pixel-center check on `20250622_000831_s4h1A.fits`:
 - * `2.387424e-12 px` max
 - * nominal-scale equivalent: `1.72e-7 mas` max
 - inverse `AZP` on the same file:
 - * `1.02e-7 mas` max
 - * `roundtrip_plane_max_error_internal=4.263256e-14`

5. Six-term `ZPN` agrees with Astropy for the PSP/WISPR test files.

- `JHV world` -> `helioprojective` -> `ZPN plane` -> `pixel`
- `ZPN plane` -> `helioprojective angles`
- implemented with `PV2_0..PV2_5`
- matches Astropy to numerical precision on:
 - `extra/test/data/psp_L3_wispr_20231227T150508_V1_1211.fits`
 - `extra/test/data/psp_L3_wispr_20231227T150704_V1_2222.fits`
- the implementation keeps only the primary monotonic branch of the radial polynomial
- inverse result on `psp_L3_wispr_20231227T150704_V1_2222.fits`:
 - `1.15e-5 mas` max
 - `roundtrip_plane_max_error_internal=4.840572e-14`

3.2 Astropy validation of JHV HPC rendering

This section compares the Python model against Astropy for the `HPC` sampling path, not only for the FITS projection formulas.

For zenithal image data, the deterministic part of the mapping is:

1. image pixel or WCS-plane coordinate
2. inverse WCS (`TAN`, `ARC`, `AZP`, `ZPN`)
3. `helioprojective angles`
4. observer-frame `HPC` ray

That intermediate `HPC` ray field is fully determined by FITS/WCS and the observer geometry.

The Python validator models this `HPC` sampling map over a bounded rendered `HPC` screen domain and compares it against Astropy.

Measured `HPC` validation results:

- COR2 (`TAN`, `20241224_194245_d4c2A.fits`) with explicit bounds report:

- raw image-footprint bounds:
 - * (-4.479846762614, 4.536935044095, -4.500971194048, 4.495966937161)
deg
- centered display bounds:
 - * (-4.536935044095, 4.536935044095, -4.536935044095, 4.536935044095)
deg
- 7.503331e-12 px max
- 2.261000e-12 px RMS
- nominal-scale equivalents: 1.10e-7 mas max, 3.32e-8 mas RMS
- HI1 (AZP, 20250622_000831_s4h1A.fts):
 - 5.002221e-12 px max
 - nominal-scale equivalent: 3.59e-7 mas max
- HI2 (AZP, 20250622_000851_s4h2A.fts):
 - 3.094152e-04 px max over the finite valid rendered domain
 - nominal-scale equivalent: 80.3 mas max
- PSP/WISPR 1211 (ZPN, psp_L3_wispr_20231227T150508_V1_1211.fits):
 - 2.902993e-10 px max
 - nominal-scale equivalent: 4.42e-5 mas max
- PSP/WISPR 2222 (ZPN, psp_L3_wispr_20231227T150704_V1_2222.fits):
 - 8.787993e-11 px max
 - nominal-scale equivalent: 1.79e-5 mas max

The centered display bounds are slightly larger than the raw image-footprint bounds because the JHV HPC screen mapping recenters the domain symmetrically about disk center and pads the shorter axis to the active display aspect.

For HI2, the finite rendered AZP domain produces the largest absolute pixel discrepancy in these recorded Python HPC comparisons because the valid image region extends close to the projection's steep outer edge, where small angular differences translate into larger source-pixel shifts.

3.3 Formal-TAN versus JHV HPC

This is not an Astropy correctness test. It compares the Python model's mappings for two different JHV display modes.

Measured comparison result:

- it compares the source pixels sampled by formal-TAN in Orthographic mode and by HPC over the same full rendered comparison frame
- native-resolution full-frame comparison between formal-TAN in Orthographic mode and HPC on the TAN samples gives:

- `sample.171.fits` (1.009 AU):
 - * 3.670609 px max, 2.401908 px RMS
 - * nominal-scale equivalents: 2.20 arcsec max, 1.44 arcsec RMS
 - * theoretical max from Appendix A: 2.19 arcsec
- `solo_L2_eui-fs1174-image_20251002T150055171_V00.fits` (0.448 AU):
 - * 2.525865 px max, 1.652452 px RMS
 - * nominal-scale equivalents: 11.22 arcsec max, 7.34 arcsec RMS
 - * theoretical max from Appendix A: 11.05 arcsec
- `20241224_194245_d4c2A.fts` (0.967 AU):
 - * 1.629534e-01 px max, 1.066298e-01 px RMS
 - * nominal-scale equivalents: 2.40 arcsec max, 1.57 arcsec RMS
 - * theoretical max from Appendix A: 2.29 arcsec

These results show:

- the `formal-TAN` path in `Orthographic` mode and `HPC` are not the same display geometry
- the maximum discrepancy is set by the on-disk part of the frame, so it remains close to the Appendix A prediction
- the visible “bulging” when switching between them is expected from that geometry difference, not from an Astropy/WCS mismatch

3.4 Simple-TAN versus JHV HPC

This is again a comparison within the Python model. Its purpose is to show whether `simple-TAN` behaves more like `HPC` or like `formal-TAN` in JHV `Orthographic` mode.

- it compares the source pixels sampled by `simple-TAN` in `Orthographic` mode and by `HPC` over the same full rendered comparison frame
- native-resolution full-frame comparison between `simple-TAN` and `HPC` gives:
 - `sample.171.fits` (1.009 AU):
 - * 2.807831e-02 px max, 8.273222e-03 px RMS
 - * nominal-scale equivalents: 16.8 mas max, 4.96 mas RMS
 - * theoretical on-disk max from Appendix B: 6.73 mas
 - `solo_L2_eui-fs1174-image_20251002T150055171_V00.fits` (0.448 AU):
 - * 1.165703e-01 px max, 3.266729e-02 px RMS
 - * nominal-scale equivalents: 518 mas max, 145 mas RMS
 - * theoretical on-disk max from Appendix B: 76.9 mas
 - * large nonzero `CRVALi` offset the solar disk from boresight and likely contribute to the higher value
 - `20241224_194245_d4c2A.fts` (0.967 AU):
 - * 1.434673e-03 px max, 3.928160e-04 px RMS
 - * nominal-scale equivalents: 21.1 mas max, 5.77 mas RMS
 - * theoretical on-disk max from Appendix B: 7.65 mas

- this confirms that `simple-TAN` is much closer to the `HPC` display geometry than `formal-TAN`, which supports its use in `Orthographic` mode when visual consistency with `HPC` is preferred.
- Appendix B gives an idealized on-disk estimate of the small-angle approximation effect. The measured full-frame maxima are larger, but they should not be read as a stronger version of the on-disk small-angle-approximation error. Outside the solar disk, `Orthographic` samples all WCS projections from the same flat image-view plane, so the off-limb part of the comparison reflects the difference between that flat `Orthographic` off-limb geometry and the `HPC` display geometry.

3.5 CAR surface-map validation

This section reports the Astropy validation results for the tested full-Sun `CAR` surface maps.

- the validated files are:
 - `extra/test/data/sunerf_map.fits`
 - `extra/test/data/syn_HMI_hmi.m_720s_2026-02-25T00-00-00_a_V1.fits`
 - `extra/test/data/syn_AIA_171_2026-01-12T00-00-00_f_V3.fits`
- all three files are `CRLN-CAR` / `CRLT-CAR` surface maps
- the correct effective scale comes from the linear WCS transform `CD`, or `diag(CDELTA1, CDELTA2) * PC`, not from bare `CDELTA`
- the Python validator keeps the original `CRLN-CAR` / `CRLT-CAR` axis types and uses the effective linear transform when constructing the Astropy comparison WCS
- JHV and Astropy agree to numerical precision
- measured results on `extra/test/data/sunerf_map.fits`:
 - full pixel-center check:
 - * `1.705303e-13 px max`
 - * nominal-scale equivalent: `3.07e-7 mas max`
 - forward sampled check:
 - * `projection_max_error_internal=2.620126e-14`
 - * `pixel_center_max_error_px=3.012701e-12`
 - inverse `CAR` on the same file:
 - * `inverse_world_max_error_deg=3.552714e-15`
 - * `roundtrip_plane_max_error_internal=2.553513e-14`
- measured results on `extra/test/data/syn_HMI_hmi.m_720s_2026-02-25T00-00-00_a_V1.fits`:
 - full pixel-center check:
 - * `2.273737e-13 px max`
 - * nominal-scale equivalent: `4.09e-7 mas max`
 - forward sampled check:
 - * `projection_max_error_internal=4.751755e-14`
 - * `pixel_center_max_error_px=1.460876e-11`

- inverse CAR on the same file:
 - * `inverse_world_max_error_deg`=5.684342e-14
 - * `roundtrip_plane_max_error_internal`=8.881784e-16
- measured results on `extra/test/data/syn_AIA_171_2026-01-12T00-00-00_f_V3.fits`:
 - full pixel-center check:
 - * `9.094947e-13 px max`
 - * nominal-scale equivalent: `6.14e-7 mas max`
 - forward sampled check:
 - * `projection_max_error_internal`=4.751755e-14
 - * `pixel_center_max_error_px`=1.460876e-11
 - inverse CAR on the same file:
 - * `inverse_world_max_error_deg`=3.552714e-15
 - * `roundtrip_plane_max_error_internal`=4.440892e-16
- these CAR cases validate the source WCS interpretation and sampling path, not the full interactive display policy for these surface maps

3.6 CEA surface-map validation

This section reports the Astropy validation results for the tested full-Sun CEA surface-map case.

- the validated file is:
 - `extra/test/data/mrzqs260301t2314c2308_169.fits`
- it is a CRLN-CEA / CRLT-CEA surface map
- the correct effective scale comes from the linear WCS transform CD, or `diag(CDELT1, CDELT2)* PC`, not from bare CDELT
- the second WCS axis is the CEA equal-area latitude coordinate, not direct angular latitude
- this file uses JHV's historical normalized missing-unit convention: CUNIT2 is absent and its second plane coordinate is stored directly as `sin(latitude) / PV2_1`
- Astropy interprets the missing unit as degrees, so the validator explicitly converts the second linear-transform row to the equivalent angular values before constructing the Astropy comparison WCS, retaining the original CRLN-CEA / CRLT-CEA axis types
- JHV and Astropy agree to numerical precision
- measured results on `extra/test/data/mrzqs260301t2314c2308_169.fits`:
 - full pixel-center check:
 - * `5.684342e-14 px max`
 - * nominal-scale equivalent: `1.02e-7 mas max`
 - forward sampled check:
 - * `projection_max_error_internal`=1.776357e-15
 - * `pixel_center_max_error_px`=1.136868e-13
 - inverse CEA on the same file:

- * `inverse_world_max_error_deg=1.136868e-13`
- * `roundtrip_plane_max_error_internal=1.776357e-15`
- this [CEA](#) case validates the source WCS interpretation and sampling path after that documented compatibility normalization, not Astropy's interpretation of the unmodified missing-unit header or the full interactive display policy for these surface maps

3.7 Dense surface-map render compare

In addition to the source-pixel and sampled WCS checks above, the Python validator includes a separate dense rendered surface-map comparison for the tested [CAR/CEA](#) files. This mode evaluates the simulated JHV sampling path and the Astropy reference on a longitude/latitude render grid at 4x the source-map width and height, with Carrington longitude 0 placed at the center of the rendered comparison image.

This is a complementary renderer-shaped check, not a replacement for the source-pixel validation numbers reported above.

- `extra/test/data/sunerf_map.fits` (CAR):
 - `pixel_center_max_error_px=1.136868e-13`
 - `pixel_center_rms_error_px=2.972981e-14`
 - `sample_max_abs_error=5.599077e-12`
- `extra/test/data/syn_HMI_hmi.m_720s_2026-02-25T00-00-00_a_V1.fits` (CAR):
 - `pixel_center_max_error_px=4.547474e-13`
 - `pixel_center_rms_error_px=1.483156e-13`
 - `sample_max_abs_error=1.178933e-10`
- `extra/test/data/syn_AIA_171_2026-01-12T00-00-00_f_V3.fits` (CAR):
 - `pixel_center_max_error_px=4.547474e-13`
 - `pixel_center_rms_error_px=1.449107e-13`
 - `sample_max_abs_error=3.185505e-10`
- `extra/test/data/mrzqs260301t2314c2308_169.fits` (CEA):
 - `pixel_center_max_error_px=1.136868e-13`
 - `pixel_center_rms_error_px=3.742207e-14`
 - `sample_max_abs_error=6.195933e-11`

These dense rendered checks stay in the same numerical range as the main [CAR/CEA](#) Astropy validation and show that the modeled JHV surface-map sampling path agrees with Astropy to numerical precision, after the documented canonicalization for the normalized missing-unit CEA file.

3.8 GLSL/Electron validation results

The Electron tests run the JHV GLES3 GLSL shader sources through a WebGL2 runner. These results measure the shader path in float32/backend arithmetic, not the double-precision mathematical model reported above.

All pixel errors in this section are source-image coordinate errors. The shader produces the texture coordinate from which the image would be sampled. The Electron runner compares that coordinate with the Python/Astropy reference coordinate and converts the difference into the original FITS image pixel grid. The values below are not framebuffer pixels, screen pixels, or visible output distortion measured on the rendered image.

Metal/ANGLE corresponds to JHV's macOS rendering backend. Among the recorded cases in the following table, the largest GLSL-to-reference difference was $1.945496\text{e-}03$ source-image pixels, on the CAR surface maps.

Metal/ANGLE source-image pixel errors:

Case	Worst source-image px
TAN AIA 171 orthographic	$4.649544\text{e-}04$
TAN EUI FSI 174 orthographic	$2.876106\text{e-}04$
TAN COR2 orthographic	$1.709235\text{e-}04$
HPC AIA 171	$1.008153\text{e-}03$
HPC COR2	$5.172269\text{e-}04$
HPC PUNCH ARC	$9.909368\text{e-}04$
HPC HI1 AZP	$2.553579\text{e-}04$
HPC HI2 AZP	$2.742185\text{e-}04$
HPC WISPR 1211 ZPN	$3.360559\text{e-}04$
HPC WISPR 2222 ZPN	$2.698873\text{e-}04$
SunERF CAR surface map	$7.295609\text{e-}04$
HMI CAR surface map	$1.945496\text{e-}03$
AIA CAR surface map	$1.945496\text{e-}03$
CEA surface map	$5.835295\text{e-}05$

The same Electron suite was also run on SwiftShader, a Vulkan software rendering implementation. It shows larger coordinate differences in HPC and non-orthographic mappings.

Recorded backend maxima for these earlier coordinate checks:

Family	Metal/ANGLE source-image px	SwiftShader source-image px
TAN/ARC/AZP/ZPN HPC render	$1.121007\text{e-}03$	$2.833492\text{e-}01$
TAN orthographic	$4.649544\text{e-}04$	$4.862770\text{e-}04$
surface maps	$1.945496\text{e-}03$	$2.964020\text{e-}02$
latitudinal	$8.688674\text{e-}04$	$5.916016\text{e-}01$

Family	Metal/ANGLE source-image px	SwiftShader source-image px
radial/rect warp	1.479215e-03	6.703000e-01

At the sampled warp boundaries, SwiftShader also disagreed with the CPU model's validity mask for a small number of pixels, whereas Metal/ANGLE had no such mismatch. These measurements precede the expanded production-fragment coverage and do not summarize its accuracy.

3.8.1 Rendered synthetic-source results

The earlier `sample_max_error` diagnostic sampled the CPU texture at the shader-produced coordinate. It therefore tested texture filtering but could not reveal that the coordinate, or the source-image validity decision, differed from the independent reference. The current sampled-texture comparison instead uses Astropy-derived coordinates for `HPC`, and the corresponding independent CPU mapping for JHV-specific geometry. `frame_sample_*` includes both-valid, shader-only, reference-only, and both-discarded pixels. `joint_sample_*` excludes visibility mismatches so that sample accuracy and visibility are reported independently. This is a source-boundary diagnostic. The separate planar-mask checks cover the display mask predicates but do not change the interpretation of these results.

Recorded results from the 128 by 128 direct-backend check:

Case	Mask differences	Metal/reference px	SwiftShader/reference px	Metal/SwiftShader px	Joint sample max	Frame sample max
PUNCH	0	9.230371e-04	2.447729e-01	2.446795e-01	2.840757e-04	2.840757e-04
ARC						
HI1 AZP	1	2.362557e-04	1.284197e-01	1.284790e-01	3.268719e-04	7.719727e-01
HI2 AZP	0	2.075942e-04	5.126388e-02	5.126953e-02	2.683401e-04	2.683401e-04
WISPR	1	2.437399e-04	1.304122e-01	1.304175e-01	3.395677e-04	4.458008e-01
1211 ZPN						
WISPR	0	2.881453e-04	8.154246e-02	8.146718e-02	3.011227e-04	3.011227e-04
2222 ZPN						

The two large frame maxima are each caused by one reference-only source-edge pixel. Over jointly visible pixels, all five rendered-value differences remain below $3.4e-4$. The source-coordinate differences remain much larger on SwiftShader, up to about 0.245 source-image pixels. Because the synthetic source varies smoothly, those coordinate errors produce much smaller rendered-value differences away from its edges.

In the earlier 512x512 runs, Metal/ANGLE had no `HPC` validity-mask differences in the five `ARC/AZP/ZPN` projection cases and remained below the $1e-3$ rendered-value threshold. SwiftShader had sparse edge-mask differences in every case:

Case	Shader-only	Reference-only
PUNCH ARC	0	12
HI1 AZP	0	10
HI2 AZP	1	2
WISPR 1211 ZPN	0	11
WISPR 2222 ZPN	0	2

Those dropped or extra pixels are real full-frame output differences: the synthetic-source maxima were between about 0.50 and 0.99. They are now reported as visibility mismatches and `frame_sample_max_error`, rather than also being misclassified as jointly valid sample errors. The current tests apply the bounded crop-edge allowance described above. `--report-validity-mismatches` instead requests a diagnostic that reports visibility differences without enforcing that limit. Away from the mask boundary, the SwiftShader texture-interpolation maximum remained about $6e-8$ after matching the CPU oracle to the uploaded [R16F](#) values.

A direct Metal/ANGLE-to-SwiftShader comparison is now part of the routine 128 by 128 suite. The earlier 512 by 512 diagnostic gave the same localization:

Case	Different mask pixels	Jointly visible sample max
PUNCH ARC	12	$2.90e-4$
HI1 AZP	10	$3.30e-4$
HI2 AZP	3	$2.77e-4$
WISPR 1211 ZPN	11	$3.55e-4$
WISPR 2222 ZPN	2	$3.11e-4$

Within this source-boundary diagnostic, all large backend-to-backend differences occur at those validity-mask pixels. The ordinary float32 CPU model still classifies the reference-only coordinates as inside the image, often by only hundredths of a source pixel. SwiftShader's accumulated coordinate error moves them across the hard `clamp_coord` boundary.

JHV estimates an image's HPC footprint from eight source-boundary points: four corners and four edge midpoints. A separate 4097-samples-per-edge audit found that this exactly reproduces the dense extrema for PUNCH [ARC](#), HI1 [AZP](#), and WISPR 1211 [ZPN](#). It misses HI2's latitude extrema by approximately $-0.188/+0.245$ degrees and WISPR 2222's minimum latitude by approximately -0.382 degrees. Those misses are a separate bounds-quality issue: in both cases the square HPC view is controlled by the much larger longitude extent, so the actual bounds supplied to the shader are unchanged and cannot explain the backend disagreement.

At 512x512, SwiftShader's four surface-map cases had matching validity masks and rendered-sample maxima below $2.5e-5$. Their larger raw coordinate differences therefore did not translate into a failure

for this smooth synthetic source at that viewport and sampling density. The corresponding Metal/ANGLE checks also stayed below the $1e-3$ rendered-value threshold.

4 Appendix A: Theoretical Orthographic vs HPC discrepancy

This appendix records a purely theoretical geometric discrepancy between:

- a straight-on orthographic projection of the visible solar surface
- a linear HPC display of the same imaged sphere

with both images scaled so that the apparent solar limb radius is the same.

This result is not derived from JHV. It follows directly from the ideal orthographic and HPC projection formulas for a unit solar sphere observed from distance D .

Assumptions:

- solar radius $R = 1$
- observer distance D expressed in solar radii
- HPC is displayed linearly in helioprojective angle
- the comparison is purely geometric and does not depend on JHV-specific rendering choices

Let γ be the heliocentric angle on the solar surface, measured from disk center toward the limb.

The visible limb occurs at:

- $\gamma_{\text{limb}} = \cos^{-1}(1/D)$

The apparent helioprojective limb angle is:

- $\theta_{\text{limb}} = \sin^{-1}(1/D)$

The orthographic and HPC radial coordinates, after equal-limb normalization, are:

- $R_{\text{ortho}}(\gamma) = \sin(\gamma)/\sqrt{1 - D^{-2}}$
- $R_{\text{hpc}}(\gamma) = \tan^{-1}(\sin(\gamma)/(D - \cos(\gamma)))/\sin^{-1}(1/D)$

The normalized discrepancy is therefore:

- $\Delta(\gamma) = R_{\text{hpc}}(\gamma) - R_{\text{ortho}}(\gamma)$

At 1 AU, with $D \approx 215.03215567$, the maximum absolute discrepancy is:

- $\max |\Delta| \approx 0.00232016$
- this occurs at $\gamma \approx 44.84^\circ$
- in angular units, this is about 2.23 arcsec
- equivalently, about 0.232% of the apparent solar limb radius

The discrepancy is positive, so the HPC image bulges outward slightly relative to the orthographic image.

The discrepancy is zero at disk center and at the limb, and peaks on a ring at about 44.8° heliocentric angle, corresponding to about 70.7% of the apparent disk radius.

The following figure shows the same discrepancy at 1 AU as a radial gradient over the normalized solar disk. The dashed ring marks the location of the maximum discrepancy.

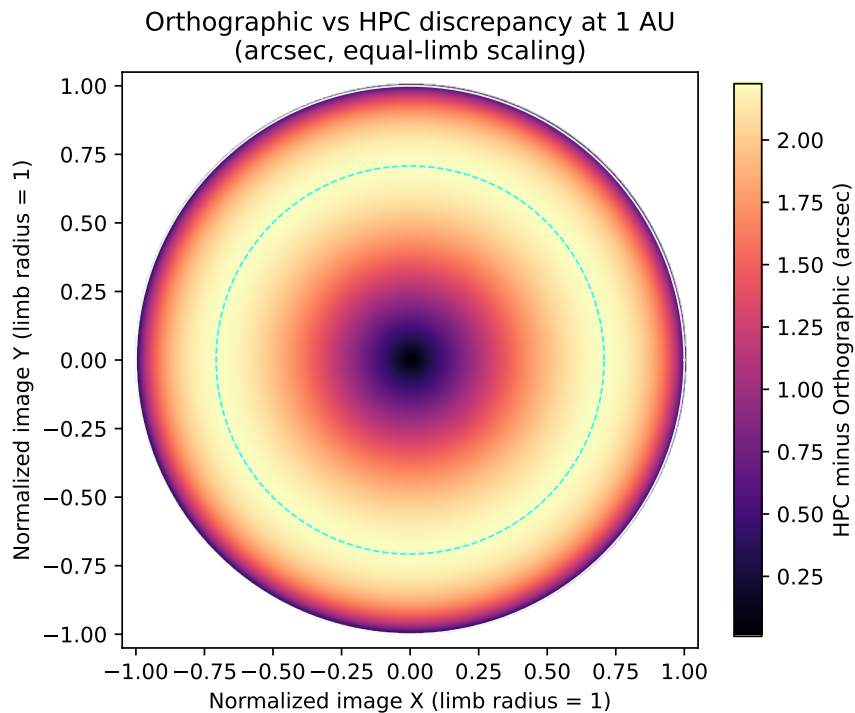


Figure 4.1: Orthographic vs HPC discrepancy across the projected solar disk at 1 AU

The following figure shows the maximum discrepancy as a function of observer distance between 0.2 AU and 1.1 AU.

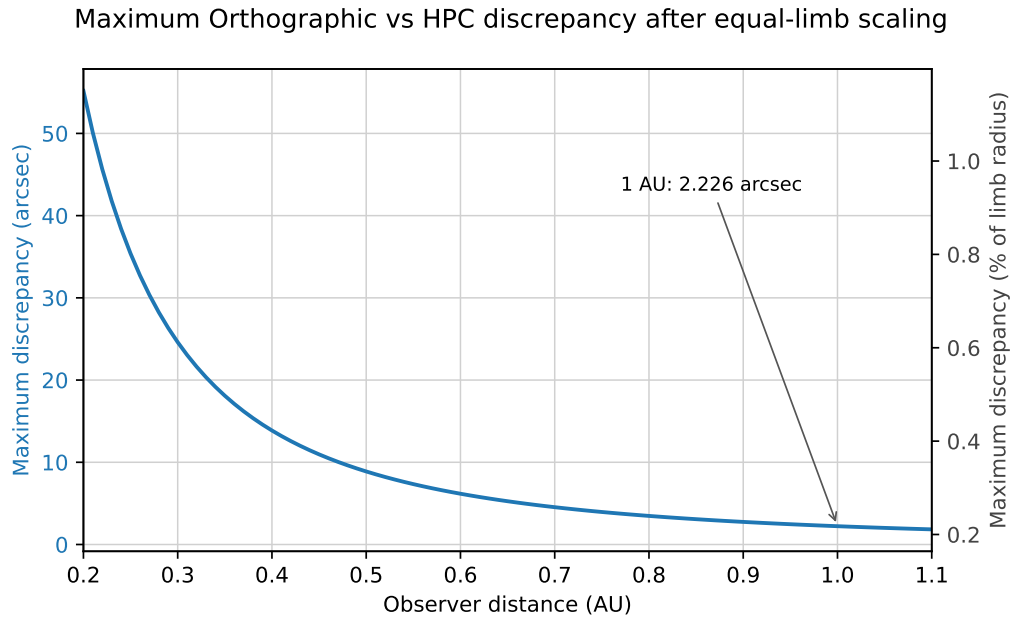


Figure 4.2: Maximum Orthographic vs HPC discrepancy versus observer distance

5 Appendix B: Idealized TAN small-angle discrepancy relative to HPC

This appendix records a different theoretical discrepancy from Appendix A. It isolates one effect only: the error introduced when **TAN** is approximated by the small-angle relation $\tan(a) \approx a$. This is a purely angular error. It is not derived from JHV. The centered solar-disk case considered below is only one example of it.

Assumptions:

- the relevant angular distance is measured from the instrument boresight
- **HPC** is displayed linearly in helioprojective angle
- the exact **TAN** plane coordinate is replaced by its small-angle approximation

Let a be the helioprojective angular distance from the instrument boresight. In linear **HPC**, the radial coordinate is proportional to a . In **TAN**, the radial coordinate on the projection plane is proportional to $\tan(a)$. The corresponding continuous discrepancy is therefore:

- $\Delta(a) = \tan(a) - a$

Unlike Appendix A, this discrepancy is monotonic in a : the function $\tan(a) - a$ increases as a increases. The discrepancy therefore grows with angular distance from the instrument boresight.

To relate this purely angular error to a solar image, consider the special case of a centered solar disk observed from distance D expressed in solar radii. Then the solar-limb angle is:

- $a_{\text{limb}} = \sin^{-1}(1/D)$

so the maximum discrepancy is:

- $\Delta_{\text{max}}(D) = \tan(\sin^{-1}(1/D)) - \sin^{-1}(1/D)$

At 1 AU, with $D \approx 215.03215567$, this gives:

- $\Delta_{\text{max}} \approx 3.35 \times 10^{-8}$ rad
- about 6.92 mas

At 0.2 AU, the same idealized discrepancy is about 865 mas.

The following figure shows this centered-disk example of the idealized maximum discrepancy as a function of observer distance between 0.2 AU and 1.1 AU.

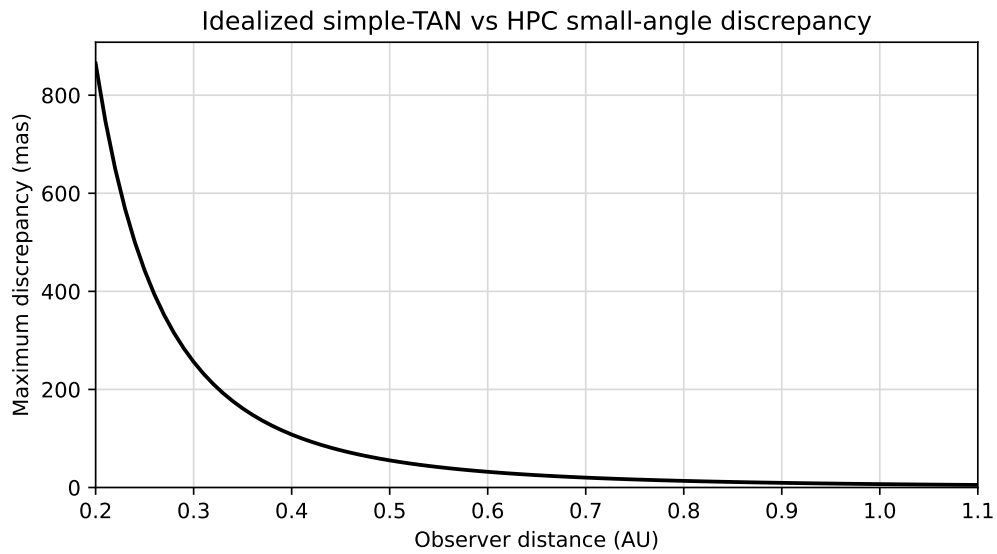


Figure 5.1: Maximum idealized TAN small-angle discrepancy relative to HPC versus observer distance

In the centered-disk example treated here, disk center coincides with the boresight, so the discrepancy increases continuously from solar disk center to the limb, where the on-disk maximum occurs. If the solar disk is offset from the boresight, the on-disk maximum is larger because part of the disk lies at larger boresight angles.

For coronagraph images, where the field of view extends to much larger angles than the solar disk, the same angular small-angle-approximation error can reach hundreds of arcseconds near the image-frame edge.